

Robocopa

Assistente Inteligente da Copa do Mundo 2026 via Telegram

Workshop de IA

Entrega Final

2026

MARQUES VINÍCIUS MELO MARTINS

Prof. Sandro Moreira

Disciplina: Tópicos em Engenharia de Software

Junho de 2026



Bot no Telegram

<https://t.me/RobocopaBot>

Repositório GitHub

<https://github.com/marquesvinicius/robocopa>

@RobocopaBot

Disponível publicamente no Telegram

1. Introdução

O **Robocopa** é um assistente de inteligência artificial desenvolvido para a plataforma Telegram, especializado exclusivamente na Copa do Mundo FIFA 2026 (realizada no Canadá, EUA e México). O projeto foi desenvolvido como trabalho final da disciplina de Tópicos em Engenharia de Software, com o objetivo de aplicar conceitos avançados de agentes de IA em um cenário real e de alto engajamento.

O bot utiliza a arquitetura **ReAct** (Reasoning and Acting), onde o modelo de linguagem raciocina explicitamente antes de agir, chamando ferramentas externas quando necessário para obter dados atualizados e precisos. Isso evita alucinações — problema crítico em domínios esportivos onde placares e escalações mudam a cada hora.

O @RobocopaBot está disponível publicamente em **t.me/RobocopaBot** e foi projetado para responder perguntas em português brasileiro, incluindo variações linguísticas naturais como *"quando joga o Brasil?"*, *"quem tá artilhando?"* ou *"me avisa o gol"*.

2. Objetivos

2.1 Objetivo Geral

Desenvolver um agente de IA conversacional capaz de fornecer informações em tempo real sobre a Copa do Mundo 2026, integrando múltiplas fontes de dados esportivos e entregando uma experiência natural e confiável ao usuário via Telegram.

2.2 Objetivos Específicos

- Implementar o loop ReAct com Gemini 2.5 Flash Lite para raciocínio e chamada de ferramentas
- Integrar APIs de dados esportivos com cadeia de fallback para garantir disponibilidade
- Prevenir alucinações em dados críticos (placares, escalações, datas)
- Entregar respostas sempre em português brasileiro, inclusive nomes de seleções
- Implementar sistema de alertas proativos com persistência Redis
- Fazer deploy em plataforma cloud (Render) com uptime contínuo e custo zero

3. Arquitetura Técnica

3.1 Loop ReAct

O núcleo do sistema é o agente ReAct implementado em **agent.py**. A cada mensagem do usuário, o modelo executa até **6 iterações** do ciclo:

[Thought] → Raciocínio interno sobre o que o usuário quer e qual ferramenta usar

[Action] → Chamada de ferramenta via Gemini Function Calling

[Obs] → Resultado da ferramenta injetado de volta no contexto

[Answer] → Resposta final em português ao usuário

Esta abordagem garante que o modelo nunca invente dados esportivos: qualquer informação factual é obrigatoriamente obtida via ferramenta.

3.2 Diagrama de Componentes

Camada	Componente	Responsabilidade
Interface	python-telegram-bot	Recebe mensagens, envia respostas, gerencia comandos e JobQueue
Agente	agent.py (ReAct)	Loop Thought→Action→Obs, máximo 6 iterações por mensagem
LLM	Gemini 2.5 Flash Lite	Raciocínio, classificação de intenção, geração de texto em PT-BR
Dados estruturados	tools/football.py	4 fontes de dados + cache compartilhado + tradução PT-BR (131 nomes)
Dados web	Tavily Search API	Busca web para notícias, canais de TV e informações não estruturadas
Persistência	Redis / arquivo JSON	Preferências do usuário e deduplicação de alertas com fallback
Notificações	notifications.py	Job periódico (5 min): pré-jogo, escalação, resultado, gol ao vivo
Deploy	Render Web Service	Hosting contínuo: health check /health + self-ping a cada 14 min

3.3 Cadeia de Fallback de Dados

Para garantir disponibilidade máxima sem depender de um único provedor, o sistema usa uma cadeia de 4 fontes de dados em ordem de prioridade:

Prior.	Fonte	Cobertura	Situação no plano Free
1ª	API-Football (api-sports)	Placares ao vivo, fixtures	Bloqueia season=2026 em endpoints principais
2ª	football-data.org	Copa completa, fixtures, standings	10 req/min — fonte principal efetiva
3ª	API-Football (por data)	Workaround: fixtures sem season=2026	Funciona mas sem placar detalhado
4ª	openfootball/worldcup.json	JSON estático da comunidade	Atualizado manualmente, sem chave de API

O cache compartilhado `_fdorg_competition_matches()` (TTL 5 min) armazena todas as partidas da Copa em disco local. Funções como próximos jogos, ao vivo, hoje e resultado filtram deste cache em Python — reduzindo chamadas externas de ~6 por sessão para **1 a cada 5 minutos**. Um flag global `_api_football_2026_blocked` é ativado no primeiro erro de plano Free e direciona chamadas subsequentes diretamente ao football-data.org, eliminando requisições desperdiçadas.

4. Funcionalidades Implementadas

4.1 Comandos Rápidos

Comando	Função
/hoje	Jogos do dia com placar ao vivo ou horário previsto
/jogos	Próximos jogos da Copa (todos os grupos)
/tabela	Classificação completa dos 12 grupos com pontuação real
/grupo A	Classificação de um grupo específico (A–L)
/aovivo	Partidas em andamento com placar em tempo real
/artilheiros	Tabela de artilheiros e assistências
/matamata	Fase eliminatória (oitavas, quartas, semi, final)
/elenco Brasil	26 jogadores convocados de uma seleção, agrupados por posição
/escalacao Noruega	Titulares da última partida (disponível ~1h antes via API paga)
/preferencias Brasil	Seguir um time para receber alertas automáticos
/alertas	Ver e gerenciar preferências de notificação
/cancelar_alertas	Remover todas as inscrições de alerta
/help	Exemplos de perguntas em linguagem natural

4.2 Linguagem Natural

Além dos comandos diretos, o usuário pode interagir livremente em português:

- "Quando joga o Brasil?" — busca próximos jogos filtrado por time
- "Quem tá artilhando?" — retorna tabela de goleadores
- "Qual foi o placar do jogo do Marrocos?" — resultado da última partida
- "Onde assistir o Brasil x Haiti?" — busca web (canal de TV)
- "Me avisa quando a Argentina jogar" — configura alerta proativo
- "Qual o retrospecto Brasil x Argentina?" — H2H histórico entre seleções

4.3 Alertas Proativos

O sistema de notificações roda a cada 5 minutos via **JobQueue** do python-telegram-bot e envia automaticamente:

- **Pré-jogo:** lembrete N minutos antes do kickoff (padrão 60 min, configurável)
- **Escalação publicada:** titulares assim que disponíveis (~1h antes do jogo)
- **Resultado final:** placar ao encerrar a partida
- **Gol ao vivo:** notificação imediata ao detectar variação no placar (opt-in)

Deduplicação via Redis garante que cada alerta seja enviado exatamente uma vez, mesmo após reinicializações do servidor. Fallback automático para arquivo JSON quando Redis está indisponível

(desenvolvimento local).

O fallback para football-data.org em `_get_fixtures_for_team()` converte o formato da API para o formato interno, garantindo que alertas funcionem mesmo com o plano Free da API-Football bloqueando a temporada 2026.

4.4 Navegação com Botões Inline

Todas as respostas de dados incluem um teclado de atalho com 4 botões: **Hoje**, **Ao vivo**, **Tabela** e **Artilheiros**. O usuário navega sem precisar digitar novos comandos.

4.5 Rate Limiting

Proteção contra spam: máximo de **10 mensagens por 60 segundos** por usuário (chat_id). Mensagens excedentes recebem aviso amigável pedindo para aguardar.

5. Tecnologias e Dependências

Tecnologia	Versão (req.)	Papel no projeto
Python	3.11+	Linguagem principal
google-genai	>=0.7.0	SDK Gemini — LLM, function calling e gerenciamento de contexto
python-telegram-bot	>=21.0	Interface Telegram: polling, handlers, JobQueue, InlineKeyboard
requests	>=2.31.0	Chamadas HTTP às APIs de dados esportivos e Tavily
redis	>=5.0.0	Persistência de preferências e deduplicação de alertas
python-dotenv	>=1.0.0	Carregamento de variáveis de ambiente (.env)
colorama	>=0.4.6	Logs coloridos no terminal (Windows-compatible)
tzdata	>=2024.1	Suporte a fuso horario de Brasilia (America/Sao_Paulo)
API-Football (api-sports.io)	v3 (externa)	Placares ao vivo e fixtures — plano Free, 100 req/dia
football-data.org	v4 (externa)	Copa 2026 completa: standings, scorers, fixtures — 10 req/min
openfootball/worldcup.json	GitHub (externo)	JSON estático da comunidade, fallback sem chave de API
Tavily Search API	v1 (externa)	Busca web via HTTP para notícias e canais de transmissao
Render.com	Free tier	Hosting do Web Service + Redis 25 MB — custo zero

6. Destaques de Implementação

6.1 Anti-Alucinação para Dados Esportivos

O principal risco de agentes de IA em contextos esportivos é inventar placares ou escalações. Três mecanismos previnem isso:

- O system prompt instrui explicitamente: *"JAMAIS invente datas de jogos, placares, escalações, artilheiros"*
- A função resultado_jogo() retorna mensagem bloqueadora quando não há dados, proibindo que o agente use web_search como substituto para placares
- O campo [Thought] obriga o modelo a raciocinar antes de agir, tornando a intenção auditável nos logs

6.2 Tradução Automática de Nomes para PT-BR

As APIs retornam nomes em inglês (*Brazil, France, Morocco*). O dicionário `_PT_BR_NAMES` com **131 entradas** e a função `_team_pt()` traduzem todos os nomes antes de exibí-los ao usuário. Aplicado em 15+ pontos do código para garantir consistência total na interface.

6.3 Flag `_api_football_2026_blocked`

Ao detectar o erro *"Free plans do not have access to this season"*, o sistema seta um flag global e pula automaticamente a API-Football nas chamadas subsequentes, indo direto ao football-data.org como segunda fonte. Isso elimina ~3 requisições desperdiçadas por sessão após o primeiro erro detectado.

6.4 Keep-Alive no Render Free Tier

O Render hiberna serviços gratuitos após 15 minutos de inatividade. O bot contorna isso com dois mecanismos combinados: um servidor HTTP leve escutando na porta configurada pelo Render (rota `/health`) e um job no JobQueue que faz self-ping a cada **14 minutos** via a variável de ambiente `RENDER_EXTERNAL_URL`, mantendo o processo ativo continuamente.

6.5 Persistência Dual Redis + Arquivo

Preferências do usuário e deduplicação de alertas usam Redis quando disponível (ambiente Render com Redis gratuito 25 MB), com fallback transparente para arquivo JSON em desenvolvimento local. O mesmo código funciona nos dois ambientes sem nenhuma configuração adicional.

6.6 Cache Compartilhado de Partidas

A função `_fdorg_competition_matches()` busca todas as partidas da Copa com um TTL de 5 minutos e armazena o resultado em disco (`worldcup_cache.json`). As funções de próximos jogos, ao vivo, hoje e resultado filtram este cache localmente em Python, reduzindo chamadas à football-data.org de ~6 por sessão para tipicamente **1 chamada a cada 5 minutos** por instância do bot.

7. Estrutura do Repositório

github.com/marquesvinicius/robocopa

```
Robocopa/
├── main.py # Entry point: handlers Telegram, rate limit, nav buttons
├── agent.py # Loop ReAct (max 6 iter) + execucao de ferramentas
├── memory.py # Historico de conversa por usuario (max 10 turnos)
├── preferences.py # Preferencias por usuario (Redis + arquivo JSON)
├── notifications.py # Alertas proativos via JobQueue (5 min)
├── storage.py # Singleton cliente Redis com lazy init
├── security.py # Validacao de inputs e sanitizacao
├── tools/
│   ├── football.py # Dados da Copa: 4 fontes, cache, traducao PT-BR
│   ├── web_search.py # Busca Tavily via HTTP
│   ├── python_exec.py # Execucao segura de Python para calculos
│   └── prompts/
│       ├── system_prompt.txt # Instrucoes do agente: roteamento e anti-alucinacao
│       ├── render.yaml # Infraestrutura como codigo (Web Service + Redis)
│       ├── requirements.txt # Dependencias Python
│       └── .env.example # Variaveis de ambiente documentadas
```

8. Resultados e Demonstração

O Robocopa foi testado durante a fase de grupos da Copa 2026 com os seguintes resultados:

Funcionalidade	Status	Observação
Jogos de hoje (/hoje)	OK	football-data.org via cache compartilhado
Proximos jogos (/jogos)	OK	Cadeia de 4 fontes, nomes em PT-BR
Placar ao vivo (/aovivo)	OK	API-Football free: filtro por data + fdorg fallback
Classificacao (/tabela)	OK	football-data.org com pontuacao real e saldo de gols
Artilheiros (/artilheiros)	OK	football-data.org /scorers
Elenco convocado (/elenco)	OK	26 jogadores agrupados por posicao (GK/DEF/MID/FWD)
Escalacao titular (/escalacao)	Parcial	API-Football free bloqueia; fallback com ultima partida + web_search
H2H historico	OK	Retrospecto com estatisticas entre duas selecoes
Alertas proativos	OK	Redis dedup + fallback fdorg garante funcionamento no plano Free
Linguagem natural PT-BR	OK	Gírias e variacoes reconhecidas pelo agente
Nomes de times em PT-BR	OK	131 entradas em _PT_BR_NAMES, aplicadas em 15+ pontos do codigo
Rate limiting	OK	10 mensagens / 60 segundos por usuario (chat_id)
Keep-alive Render	OK	Self-ping a cada 14 min via JobQueue + endpoint /health
Deploy Render free tier	OK	Web Service + Redis = R\$0/mes

9. Conclusão

O projeto demonstrou na prática como a arquitetura ReAct resolve o principal problema de agentes de IA em domínios factuais: a tendência de inventar informações. Ao obrigar o modelo a raciocinar explicitamente e buscar dados via ferramentas, o Robocopa entrega respostas confiáveis mesmo com as restrições do plano gratuito das APIs esportivas.

A cadeia de fallback de 4 fontes garante disponibilidade superior a qualquer fonte individual. O cache compartilhado reduz o consumo de quota em aproximadamente 80%, permitindo operação sustentável dentro dos limites gratuitos de football-data.org (10 requisições por minuto).

Do ponto de vista de engenharia, o projeto consolidou habilidades em: design de agentes autônomos com function calling, integração de múltiplas APIs com tratamento robusto de erros, deploy cloud com custo zero, persistência em ambientes efêmeros via Redis, e prevenção ativa de alucinações em domínios críticos.

O bot está disponível publicamente em **t.me/RobocopaBot**. O código-fonte completo está em **github.com/marquesvinicius/robocopa**.

Bot Telegram: t.me/RobocopaBot

Repositorio: github.com/marquesvinicius/robocopa

Contato: marquesviniciusmelomartins@gmail.com

UNIRV — Universidade de Rio Verde | Tópicos em Engenharia de Software | 2026

MARQUES VINICIUS MELO MARTINS | Prof. Sandro Moreira